

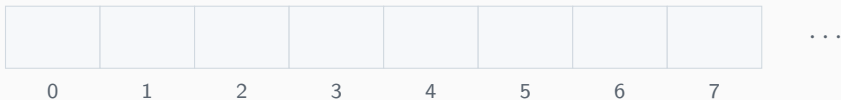
Day 2: Random Topics

Computational Integer Optimization · SoSe 2026

Gennesaret Tjusila

A brief excursion on arrays

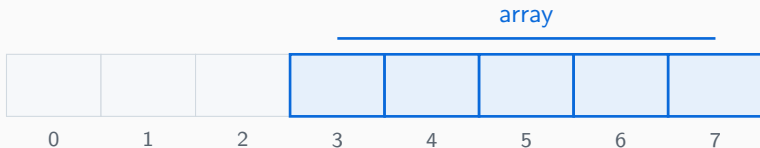
When running a program, you can imagine a contiguous block of memory with addresses.



The number below each box is its address.

What is an array?

An array is a contiguous sequence of elements in memory.

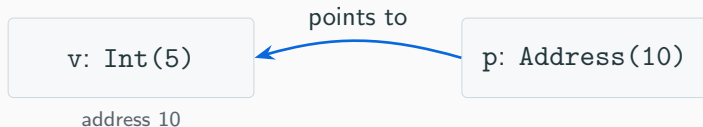


In C, an array commonly decays to a pointer to its first element. The size is not carried with that pointer.

Storing an address with a pointer

In C, we can store the address of a variable using a pointer.

```
int v = 5;    // an integer variable containing the value 5
int* p = &v;  // a pointer containing the address of v
```



Returning a value through an argument

Yesterday, we learned how C libraries can return a value through an output argument.

```
#include <iostream>

int fibonacci_10(int a, int b, int* c)
{
    if (c == nullptr)
        return 0;

    for (int i = 2; i <= 10; ++i) {
        int next = a + b;
        a = b;
        b = next;
    }

    *c = b;
    return 1;
}
```

Passing the output address

The type of `c` is `int*`. We pass the address where the function should write its result.

```
int main()
{
    int value;

    if (fibonacci_10(0, 1, &value))
        std::cout << value << '\n'; // 55

    return 0;
}
```

The function returns whether it succeeded through its return value, and returns the Fibonacci number through `c`.

Why does SCIPcreate take SCIP**?

After SCIP creates the SCIP object, it returns the address of that object through an output argument:

```
SCIP_RETCODE SCIPcreate(SCIP** scip);
```

Subsequent calls only need the address of the existing object:

```
SCIP_RETCODE SCIPincludeDefaultPlugins(SCIP* scip);
```

- SCIP* : the address of a SCIP object
- SCIP** : the address of a variable in which SCIP can write a SCIP*

Can we recover the variable pointers?

Yesterday, after creating our variables, we stored their pointers. Suppose we lose them. Can we recover the variable pointers from SCIP?

- We have a type: `SCIP_VAR` .
- Since it is stored in heap memory, SCIP has a pointer to it: `SCIP_VAR*` .
- SCIP stores a list of these pointers as an array.
- An array is exposed as a pointer to its first element: `SCIP_VAR**` .

Indeed, this is what the SCIP API exposes:

```
SCIP_VAR** SCIPgetVars(SCIP* scip)
```


How many variables are in the array?

SCIP also tells us the number of variables:

```
int SCIPgetNVars(SCIP* scip);
```

Therefore, the following is legal:

```
// scip is a SCIPPtr
int n = SCIPgetNVars(scip.get());
SCIP_VAR** vars = SCIPgetVars(scip.get());

for (int i = 0; i < n; ++i) {
    // Get the objective coefficient, e.g. SCIPvarGetObj(vars[i])
}
```

The pointer and size are separate

This introduces a problem: we need to track the pointer and its size separately.

```
void do_something_with_array_of_var(  
    SCIP_VAR** arr_ptr, int arr_size)  
{  
    // ...  
}  
  
// scip is a SCIPPtr  
int n = SCIPgetNVars(scip.get());  
SCIP_VAR** vars = SCIPgetVars(scip.get());  
  
do_something_with_array_of_var(vars, n);
```

Every function that receives the array must also receive the correct size.

Wrapping the C-style array with `std::span`

C++ makes our lives easier by allowing us to wrap a C-style array with an `std::span`.

```
void do_something_with_array_of_var(std::span<SCIP_VAR*> vars)
{
    for (int i = 0; i < std::ssize(vars); ++i) {
        // Do something with vars[i]
    }

    for (auto var : vars) {
        // Do a range-based loop
    }
}
```

A pointer and size become one view

The span wraps the pointer to the first element together with the size.

```
// std::span<type> name(pointer_to_first_element, size);  
std::span<SCIP_VAR*> vars(  
    SCIPgetVars(scip.get()),  
    SCIPgetNVars(scip.get())  
);  
  
do_something_with_array_of_var(vars);
```

- The span does not own the SCIP variables.
- It gives us one object through which we can access both the elements and the size.

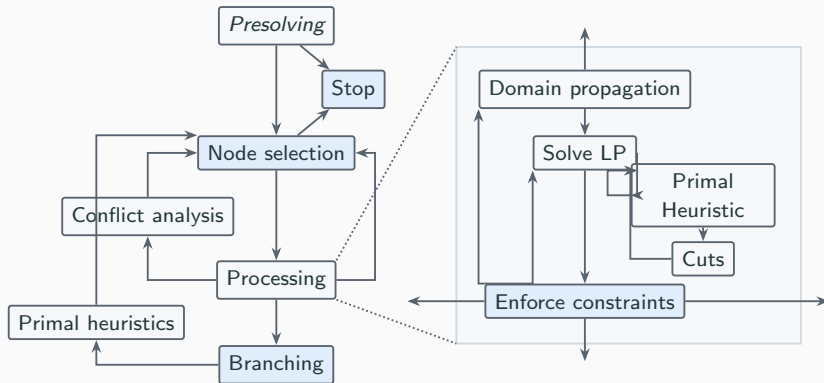
A callback is a function—the *callee*—that is called from another function—the *caller*—under two agreements:

1. when the callee will be called; and
2. the callee's signature: its return type and argument types.

The agreed signature allows the caller to pass relevant information to the callee through arguments.

We will avoid the implementation details inside the calling function.

The SCIP solving loop



All parts of the loop are plugins.

Adding a heuristic

At a particular position in the loop, SCIP calls all heuristics registered to run at that position.

For our heuristic, we choose:

```
SCIP_HEURTIMING_DURINGLPLOOP
```

We implement the heuristic as a SCIP plugin and register its callback with SCIP.

Declaring the heuristic plugin

```
#pragma once

#include <objscip/objheur.h>
#include <scip/type_heur.h>
#include <scip/type_retcodes.h>

class ZIRoundHeur : public scip::ObjHeur {
public:
    ZIRoundHeur(SCIP* scip);
    ~ZIRoundHeur() = default;

    SCIP_DECL_HEUREXEC(scip_exec) override;
};
```


Choosing the heuristic settings

```
ZIRoundHeur::ZIRoundHeur(SCIP* scip)
: scip::ObjHeur(
    scip,
    "ziround",           // HEUR_NAME
    "ZI Rounding Heuristic", // HEUR_DESC
    'z',                 // HEUR_DISPCHAR
    99999,               // HEUR_PRIORITY
    1,                   // HEUR_FREQ
    0,                   // HEUR_FREQOFS
    -1,                  // HEUR_MAXDEPTH
    SCIP_HEURTIMING_DURINGLPOOP, // HEUR_TIMING
    false)               // HEUR_USESUBSCIP
{ }
```

Registering the callback with SCIP

In `main`, we include the heuristic plugin in SCIP:

```
CALL_CHECK(SCIPincludeObjHeur(  
    scip.get(),  
    new ZIRoundHeur(scip.get()),  
    TRUE  
));
```

SCIP can now call our heuristic at its registered timing.

The callback contract

The contract is defined in the SCIP source code:

```
// type_heur.h
#define SCIP_DECL_HEUREXEC(x) \
    SCIP_RETCODE x(SCIP* scip, SCIP_HEUR* heur, \
        SCIP_HEURTIMING heurtiming, SCIP_Bool nodeinfeasible, \
        SCIP_RESULT* result)
```

Expanding `SCIP_DECL_HEUREXEC(scip_exec)` gives:

```
SCIP_RETCODE scip_exec(
    SCIP* scip,
    SCIP_HEUR* heur,
    SCIP_HEURTIMING heurtiming,
    SCIP_Bool nodeinfeasible,
    SCIP_RESULT* result);
```

Two important parameters

`SCIP* scip` This pointer is borrowed from SCIP. Instead of using `scip.get()` , we use `scip` inside the callback. SCIP takes care of freeing it; we do not need to do so.

`SCIP_RESULT* result` Through this output argument, we tell SCIP whether the heuristic could not run, did not find a solution, or found a solution.

Typical result values include:

`SCIP_DIDNOTRUN`

`SCIP_DIDNOTFIND`

`SCIP_FOUNDSOL`

Plan for the day and the rest of the week

We will solve multiple-knapsack instances.

The instances are provided as MPS files in:

```
mps_files/
```

We will implement heuristics and test how they affect the performance of our MIP solver.

Multiple-knapsack instances

The instances have the form

$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & Ax \leq b, \\ & A \in \mathbb{Z}^{m \times n}, \\ & a_{ij} \geq 0, \quad b_i \geq 0, \\ & x_i \in \{0, 1\}. \end{aligned}$$

Building and running the benchmark

To test the performance of our MIP solver, run:

```
cmake --preset debug  
cmake --build build  
./benchmark 1
```

or:

```
cmake --preset release  
cmake --build build  
./benchmark 1
```

The benchmark reports time, node count, and the primal-dual integral.

Debug or release?

`debug` SCIP runs internal checks to minimize the probability of doing something wrong. This makes SCIP slower. A debugger can also be used here.

`release` SCIP is faster, but does not run these checks. I would not recommend debugging here.

Choosing a test set

There are four test sets: 1, 2, 3, and 4.

- Test set 1 is the easiest; test set 4 is the hardest.
- I recommend test set 1 for a quick check.
- Test set 2 or 3 should give you a good idea of the strength of your implementation.

```
./benchmark 1
```

```
./benchmark 2
```

```
./benchmark 3
```

```
./benchmark 4
```

Plan for today

We will begin by implementing the ZI rounding heuristic. You can then proceed to:

- improve the implementation—I will give tips later; or
- implement another heuristic that you come up with yourself.

ZI rounding algorithm

1. Get the LP solution.
2. For each integer variable with a fractional solution:
 - 2.1 check how far the variable can be moved up or down without violating any constraint;
 - 2.2 choose how to round the variable; and
 - 2.3 update the row activities.
3. Continue with the next fractional integer variable.

Choosing how to round a variable

For each fractional integer variable:

- If the variable can be rounded up or down, round it in the best objective direction.
- If the variable can be rounded in only one direction, round it in that direction.
- If the variable cannot be rounded, check whether moving it up, moving it down, or leaving it unchanged makes the fractionality of the solution smallest.

ZI rounding pseudocode

```
solution <- current LP solution

for each fractional integer variable  $x_i$  in solution:
    can_round_up    <- check constraints
    can_round_down  <- check constraints

    if can_round_up and can_round_down:
        direction <- best objective direction
    else if can_round_up or can_round_down:
        direction <- the feasible direction
    else:
        direction <- up, down, or stay, whichever makes
                     the solution's fractionality smallest

move  $x_i$  in direction
update row activities
```